

MoneyMatters

A Full-Stack Personal Finance Management Application

Project Report

Name	Role
Aryan Vibhuti	Backend & Architecture
Rajdeep Sanyal	Backend & API Design
Piyush Yadav	Frontend Development
Meet Ahuja	Frontend Development

System Design Capstone Project

April 29, 2026

Contents

1	Problem Statement and Solution Approach	3
1.1	Problem Statement	3
1.2	Solution Approach	3
1.3	Technology Choices	4
2	System Design and Architecture	5
2.1	Architectural Overview	5
2.2	Backend Layer Responsibilities	5
2.3	System Design Optimisations	6
2.3.1	Repository Pattern for Scalability	6
2.3.2	Centralised Error Handling	6
2.3.3	Stateless JWT Authentication	7
2.3.4	Dynamic Analytics Computation	7
2.3.5	Denormalised Category Storage	7
2.3.6	Frontend Architecture	7
3	Object-Oriented Programming Concepts	8
3.1	Encapsulation	8
3.1.1	Service Classes	8
3.1.2	Repository Classes	9
3.1.3	ApiError and ApiResponse	9
3.2	Abstraction	9
3.2.1	Repository Abstraction	9
3.2.2	asyncHandler Abstraction	9
3.3	Inheritance	10
3.3.1	ApiError extends Error	10
3.3.2	Mongoose Document Interfaces	10
3.4	Polymorphism	10
3.4.1	Static Controller Methods as Polymorphic Handlers	10
3.4.2	ApiResponse Generic Type	11
3.4.3	Error Middleware Polymorphism	11
4	SOLID Principles	12
4.1	Single Responsibility Principle (SRP)	12
4.2	Open/Closed Principle (OCP)	12
4.2.1	Service Layer Extension	12
4.2.2	Middleware Chain	13
4.2.3	ApiResponse Generic	13
4.3	Liskov Substitution Principle (LSP)	13

4.3.1	ApiError as Error Substitute	13
4.3.2	Repository Substitution in Tests	13
4.4	Interface Segregation Principle (ISP)	14
4.5	Dependency Inversion Principle (DIP)	14
4.5.1	Constructor Injection	14
4.5.2	DashboardService Dual Dependency	14
5	Design Patterns	16
5.1	Repository Pattern	16
5.1.1	Intent	16
5.1.2	Implementation in MoneyMatters	16
5.1.3	Benefits Realised	17
5.2	Middleware Chain Pattern (Chain of Responsibility)	17
5.2.1	Intent	17
5.2.2	Implementation in MoneyMatters	17
5.2.3	Benefits Realised	18
5.3	Decorator Pattern (asyncHandler)	18
5.3.1	Intent	18
5.3.2	Implementation in MoneyMatters	18
5.4	Factory / Composition Root Pattern	19
5.4.1	Intent	19
5.4.2	Implementation in MoneyMatters	19
6	UML Diagrams	20
6.1	Class Diagram	20
6.2	Use Case Diagram	21
6.3	Sequence Diagram — Close Month Workflow	22
6.4	Entity-Relationship Diagram	23
7	Test Cases and Results	24
7.1	Testing Strategy	24
7.2	AuthService Tests	24
7.3	TransactionService Tests	24
7.4	MonthlyService Tests	25
7.5	DashboardService Tests	25
7.6	Controller Tests	26
7.7	Middleware Tests	26
7.8	Test Summary	27
8	API Reference	28
8.1	Authentication — /api/auth	28
8.2	Transactions — /api/transactions	28
8.3	Monthly Plan — /api/monthly-plan	28
8.4	Dashboard — /api/dashboard	29
9	Security Design	30
10	Conclusion	31

Chapter 1

Problem Statement and Solution Approach

1.1 Problem Statement

Modern individuals—particularly freelancers and gig-economy workers—face a persistent challenge in managing personal finances. The core difficulties are:

1. **Irregular income tracking.** Freelancers receive payments from multiple clients at unpredictable intervals. Without a unified log, total monthly income is unknown until bank statements are reviewed.
2. **Unstructured expense recording.** Day-to-day spending across categories such as food, travel, and utilities is rarely tracked in real time, making it impossible to identify overspending patterns.
3. **Absence of goal-oriented savings.** Most individuals set informal savings targets but have no mechanism to measure whether those targets are being met at the end of a month.
4. **Lack of visual financial insight.** Without aggregated analytics, users cannot quickly assess their financial health or compare income against expenditure.

1.2 Solution Approach

MoneyMatters addresses these problems through a structured, full-stack web application built on a layered architecture. The solution is decomposed into four functional pillars:

Transaction Logging

A unified `Transaction` model captures both income and expense entries, differentiated by a `type` field ("income" or "expense"). Each transaction carries an amount, category, date, and a reference to the owning user.

Monthly Goal Management

A `MonthlyPlan` entity allows users to set a savings target for a specific month and year. At the end of the month, the user explicitly closes the month, triggering a server-side computation that derives `achievedSavings` from transaction data and evaluates whether the goal was met.

Dashboard Analytics

A dedicated `DashboardService` aggregates transaction data on demand to produce KPI cards (earnings, spendings, cumulative savings), a category-level expense breakdown, an income-vs-expense distribution, and a goal-progress indicator.

Secure Authentication

JWT-based stateless authentication protects all non-public endpoints. Passwords are hashed with `bcrypt` before persistence. The frontend persists the token in `localStorage` and injects it into every outbound request via an Axios interceptor.

1.3 Technology Choices

Component	Technology	Rationale
Backend	Node.js ≥ 18	Non-blocking I/O; ideal for REST APIs
Run-time		
API Framework	Express 5	Minimal, well-understood; async error propagation
Language	TypeScript 6	Static typing reduces runtime errors; interfaces model domain
Database	MongoDB 9 / Mongoose 9	Schema-flexible; natural JSON document storage
Auth	JWT + bcrypt	Stateless; no server-side session storage required
Frontend	React 19 + Vite 8	Component model maps cleanly to UI features
Styling	Tailwind CSS v4	Utility-first; rapid, consistent UI development
Charts	Recharts 3	Declarative React chart components
Testing	Jest + ts-jest	Industry-standard; excellent TypeScript support

Chapter 2

System Design and Architecture

2.1 Architectural Overview

MoneyMatters follows a **layered monorepo architecture** with a strict separation of concerns. The backend implements a five-layer stack; the frontend is a context-driven React SPA. The two halves communicate exclusively over HTTP REST.

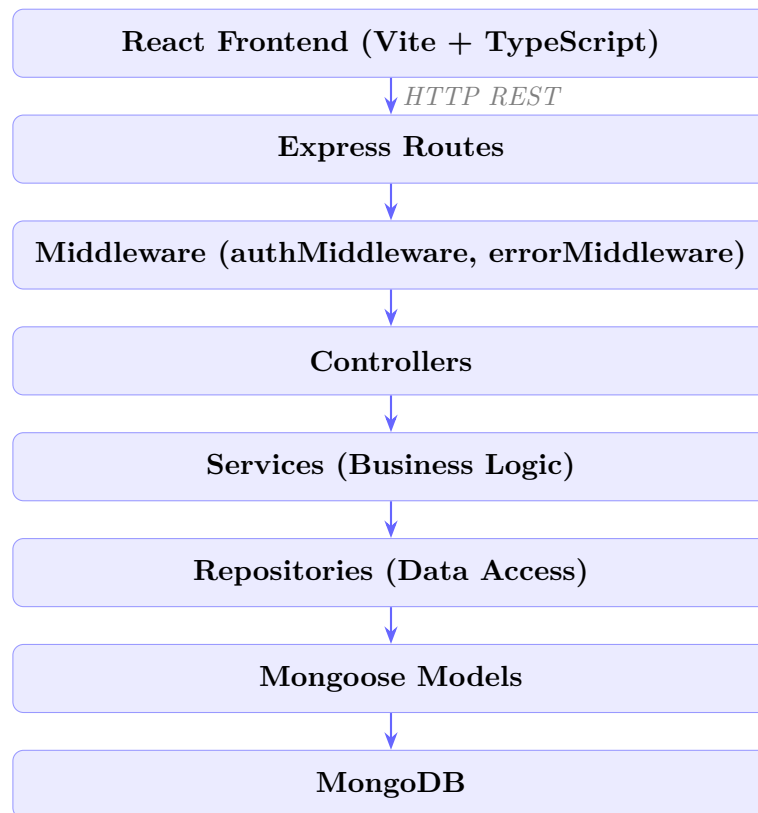


Figure 2.1: Full-stack layered architecture of MoneyMatters

2.2 Backend Layer Responsibilities

Layer	Responsibility
Routes	Define HTTP endpoints; apply middleware; delegate to controllers
Middleware	JWT validation (<code>authMiddleware</code>); centralised error handling (<code>errorMiddleware</code>)
Controllers	Parse HTTP request; call service methods; wrap result in <code>ApiResponse</code>
Services	Encapsulate all business logic; throw <code>ApiError</code> on violations
Repositories	Abstract all Mongoose queries; single point of database interaction
Models	Mongoose schemas with TypeScript interfaces (<code>IUser</code> , <code>ITransaction</code> , etc.)
Utils	<code>ApiError</code> , <code>ApiResponse</code> , <code>asyncHandler</code>

2.3 System Design Optimisations

2.3.1 Repository Pattern for Scalability

All database queries are isolated inside repository classes (`UserRepository`, `TransactionRepository`, `MonthlyPlanRepository`, `CategoryRepository`). This means:

- The database engine (MongoDB) can be swapped without touching service or controller code.
- Repositories can be independently mocked in unit tests, enabling fast, database-free test execution.
- Query optimisations (indexes, projections) are localised to one file.

The `MonthlyPlan` collection carries a compound unique index on (`userId`, `month`, `year`), enforced at the Mongoose schema level:

```

1 monthlyPlanSchema.index(
2   { userId: 1, month: 1, year: 1 },
3   { unique: true }
4 );
```

Listing 2.1: Compound unique index on `MonthlyPlan`

This prevents duplicate plans at the database layer, providing a second line of defence beyond the service-level guard.

2.3.2 Centralised Error Handling

A single `errorMiddleware` function intercepts all errors propagated through Express's `next(err)` mechanism. The `asyncHandler` higher-order function wraps every async route handler, eliminating repetitive `try/catch` blocks and ensuring no unhandled promise rejection escapes:

```

1 export const asyncHandler =
```

```
2 (fn: AsyncRouteHandler) =>
3 (req: Request, res: Response, next: NextFunction) => {
4   Promise.resolve(fn(req, res, next)).catch(next);
5 }
```

Listing 2.2: asyncHandler – eliminates try/catch boilerplate

2.3.3 Stateless JWT Authentication

Authentication is stateless: the server issues a signed JWT on login and validates it on every subsequent request. No session store is required, making horizontal scaling trivial—any server instance can validate any token without shared state.

2.3.4 Dynamic Analytics Computation

Dashboard data is computed on demand from raw transaction records rather than being pre-aggregated. This eliminates data duplication and ensures the dashboard always reflects the current state of the database. The trade-off (slightly higher per-request computation) is acceptable at the scale of a personal finance application.

2.3.5 Denormalised Category Storage

`Transaction.category` stores the category name as a plain string rather than a foreign key reference. This avoids a join on every transaction read and simplifies query logic, at the cost of referential integrity. A separate `Category` collection exists as a lookup/seed table but is not enforced via foreign key constraints.

2.3.6 Frontend Architecture

The React frontend uses a **Context + Provider** pattern for global authentication state (`AuthContext`). An Axios interceptor automatically attaches the `Authorization: Bearer <token>` header to every outbound request, decoupling authentication concerns from individual page components. Route protection is handled by a `ProtectedRoute` component that reads from `AuthContext` and redirects unauthenticated users to `/login`.

Chapter 3

Object-Oriented Programming Concepts

3.1 Encapsulation

Encapsulation is the bundling of data and the methods that operate on that data within a single unit, hiding internal implementation details from the outside world.

3.1.1 Service Classes

Each service class encapsulates its business logic and exposes only a clean public interface. The AuthService hides the details of password hashing and JWT signing behind register() and login() methods:

```
1 export class AuthService {
2   constructor(private userRepo: UserRepository) {}
3
4   async register(data: { name: string; email: string; password:
5     string }) {
6     const existing = await this.userRepo.findByEmail(data.email);
7     if (existing) throw new ApiError(400, "User already exists");
8     const hashedPassword = await bcrypt.hash(data.password, 10);
9     return this.userRepo.create({ ...data, password: hashedPassword
10       });
11   }
12
13   async login(email: string, password: string) {
14     const user = await this.userRepo.findByEmail(email);
15     if (!user) throw new ApiError(404, "User not found");
16     const isMatch = await bcrypt.compare(password, user.password);
17     if (!isMatch) throw new ApiError(401, "Invalid credentials");
18     const token = jwt.sign(
19       { userId: user._id },
20       process.env.JWT_SECRET as string,
21       { expiresIn: "1d" }
22     );
23     return { user, token };
24   }
25 }
```

Listing 3.1: AuthService encapsulates bcrypt and JWT internals

Callers of `AuthService` have no knowledge of bcrypt rounds, JWT expiry, or the underlying repository implementation.

3.1.2 Repository Classes

Repository classes encapsulate all Mongoose query logic. The `TransactionRepository` hides the `ObjectId` conversion and date range construction from its consumers:

```
1  async findByUserAndMonth(  
2    userId: string, month: number, year: number  
3  ): Promise<ITransaction[]> {  
4    const start = new Date(year, month - 1, 1);  
5    const end   = new Date(year, month, 0, 23, 59, 59);  
6    return Transaction.find({  
7      userId: new Types.ObjectId(userId),  
8      date: { $gte: start, $lte: end },  
9    });  
10 }
```

Listing 3.2: `TransactionRepository` encapsulates query construction

3.1.3 ApiError and ApiResponse

`ApiError` encapsulates an HTTP status code alongside an error message, extending the native `Error` class. `ApiResponse` encapsulates the success flag, message, and data payload into a single reusable wrapper.

3.2 Abstraction

Abstraction exposes only the essential features of an object while hiding unnecessary implementation details.

3.2.1 Repository Abstraction

Services depend on repository *interfaces* (TypeScript structural typing) rather than concrete Mongoose implementations. In tests, the repository is replaced with a plain mock object that satisfies the same interface:

```
1  // In tests -- a plain object satisfies the structural interface  
2  const mockUserRepo = {  
3    create: jest.fn(),  
4    findByEmail: jest.fn(),  
5    findById: jest.fn(),  
6  };  
7  const service = new AuthService(mockUserRepo as any);
```

Listing 3.3: Service depends on repository interface, not implementation

3.2.2 asyncHandler Abstraction

The `asyncHandler` utility abstracts the promise-rejection-to-Express error-propagation pattern. Controllers are written as if they are synchronous; the wrapper handles the async plumbing invisibly.

3.3 Inheritance

3.3.1 ApiError extends Error

ApiError inherits from the built-in Error class, gaining stack trace capture and the message property while adding a statusCode field:

```
1 export class ApiError extends Error {
2   statusCode: number;
3
4   constructor(statusCode: number, message: string) {
5     super(message);           // inherit Error.message
6     this.statusCode = statusCode;
7     Error.captureStackTrace(this, this.constructor);
8   }
9 }
```

Listing 3.4: ApiError inherits from Error

The errorHandler uses instanceof ApiError to distinguish application errors from unexpected runtime errors, demonstrating polymorphic type checking.

3.3.2 Mongoose Document Interfaces

Each model interface extends Mongoose's Document:

```
1 export interface IUser extends Document {
2   name: string;
3   email: string;
4   password: string;
5 }
```

Listing 3.5: IUser extends Document

This inheritance grants every model instance Mongoose document methods (save(), toObject(), etc.) while adding domain-specific typed fields.

3.4 Polymorphism

3.4.1 Static Controller Methods as Polymorphic Handlers

All controller methods share the same Express handler signature (req, res, next). Express dispatches to the correct handler polymorphically based on the HTTP method and route path:

```
1 router.post("/register", AuthController.register);
2 router.post("/login", AuthController.login);
3 router.post("/", TransactionController.create);
4 router.get("/", TransactionController.getAll);
5 router.delete("/:id", TransactionController.delete);
```

Listing 3.6: Polymorphic route dispatch

3.4.2 ApiResponse Generic Type

`ApiResponse<T>` is a generic class. The same class is instantiated with different type parameters across controllers, providing type-safe polymorphic response wrapping:

```
1 // In AuthController
2 res.status(201).json(new ApiResponse(201, user, "User registered"));
3
4 // In TransactionController
5 res.json(new ApiResponse(200, transactions, "Transactions fetched"));
6
7 // In DashboardController
8 res.json(new ApiResponse(200, dashboardData, "Dashboard data
   fetched"));
```

Listing 3.7: Generic ApiResponse used polymorphically

3.4.3 Error Middleware Polymorphism

The `errorMiddleware` handles both `ApiError` and generic `Error` instances through runtime type checking, producing different HTTP responses from the same function:

```
1 export const errorMiddleware = (
2   err: Error, _req: Request, res: Response, _next: NextFunction
3 ) => {
4   let statusCode = 500;
5   let message     = "Internal Server Error";
6
7   if (err instanceof ApiError) { // polymorphic branch
8     statusCode = err.statusCode;
9     message     = err.message;
10  }
11
12  res.status(statusCode).json({ success: false, message });
13 };
```

Listing 3.8: Polymorphic error handling

Chapter 4

SOLID Principles

4.1 Single Responsibility Principle (SRP)

A class should have only one reason to change.

Every class in MoneyMatters has a single, well-defined responsibility:

Class	Single Responsibility
UserRepository	Persist and retrieve User documents
AuthService	Register users and authenticate credentials
AuthController	Parse HTTP auth requests; format HTTP responses
TransactionService	Validate and manage transaction lifecycle
DashboardService	Aggregate financial analytics from raw data
MonthlyService	Manage monthly plan creation and month-close logic
authMiddleware	Validate JWT tokens on incoming requests
errorMiddleware	Translate errors into structured HTTP responses
asyncHandler	Wrap async handlers for promise rejection forwarding
ApiError	Represent application-level HTTP errors
ApiResponse	Represent structured HTTP success responses

For example, **AuthController** does not contain any password-hashing logic (that belongs to **AuthService**), and **AuthService** does not contain any Mongoose queries (those belong to **UserRepository**).

4.2 Open/Closed Principle (OCP)

Software entities should be open for extension but closed for modification.

4.2.1 Service Layer Extension

New business operations can be added to the system by creating new service classes without modifying existing ones. For example, a future **BudgetService** or **ReportService** can be introduced alongside **DashboardService** without touching any existing code.

4.2.2 Middleware Chain

Express middleware is composed as a chain. New cross-cutting concerns (rate limiting, request logging, input sanitisation) can be added to `app.ts` without modifying any existing middleware or controller:

```
1 app.use(express.json());
2 app.use("/api/auth", authRoutes);
3 app.use("/api/transactions", transactionRoutes);
4 app.use("/api/monthly-plan", monthlyRoutes);
5 app.use("/api/dashboard", dashboardRoutes);
6 app.use(errorMiddleware); // new middleware can be inserted above
   this line
```

Listing 4.1: Middleware chain is open for extension

4.2.3 ApiResponse Generic

`ApiResponse<T>` is closed for modification but open for extension via its generic type parameter. Any new data shape can be wrapped without changing the class itself.

4.3 Liskov Substitution Principle (LSP)

Subtypes must be substitutable for their base types.

4.3.1 ApiError as Error Substitute

`ApiError` extends `Error`. Anywhere an `Error` is expected—such as the `errorMiddleware` signature—an `ApiError` can be passed without breaking the contract. The middleware correctly handles both:

```
1 export const errorMiddleware = (err: Error, ...) => {
2   // Works whether err is Error or ApiError
3   if (err instanceof ApiError) { ... }
4   };
```

Listing 4.2: `ApiError` is substitutable for `Error`

4.3.2 Repository Substitution in Tests

In every test suite, the concrete repository class is replaced with a mock object that satisfies the same structural interface. The service under test behaves identically regardless of whether it receives the real repository or the mock—a direct demonstration of LSP:

```
1 const mockUserRepo = {
2   create: jest.fn(),
3   findByEmail: jest.fn(),
4   findById: jest.fn(),
5 };
6 // AuthService works identically with mock or real UserRepository
7 const service = new AuthService(mockUserRepo as any);
```

Listing 4.3: Mock repository substitutes real repository

4.4 Interface Segregation Principle (ISP)

Clients should not be forced to depend on interfaces they do not use.

Each repository exposes only the methods required by its consumers. The `CategoryRepository` exposes a single method because that is all that is needed:

```
1 export class CategoryRepository {
2   async findAll(): Promise<ICategory []> {
3     return Category.find();
4   }
5 }
```

Listing 4.4: `CategoryRepository` – minimal interface

The `MonthlyPlanRepository` exposes exactly the five methods consumed by `MonthlyService` and `DashboardService`—no more. Mock objects in tests mirror this minimal surface, confirming that no consumer depends on methods it does not use.

4.5 Dependency Inversion Principle (DIP)

High-level modules should not depend on low-level modules. Both should depend on abstractions.

4.5.1 Constructor Injection

Every service receives its repository dependency through constructor injection rather than instantiating it internally:

```
1 export class MonthlyService {
2   constructor(
3     private monthlyRepo: MonthlyPlanRepository,
4     private transactionRepo: TransactionRepository,
5   ) {}
6 }
```

Listing 4.5: DIP via constructor injection in `MonthlyService`

`MonthlyService` (high-level) depends on the repository abstraction, not on a concrete Mongoose call. The concrete implementation is injected at the composition root (the controller file):

```
1 const service = new MonthlyService(
2   new MonthlyPlanRepository(),
3   new TransactionRepository()
4 );
```

Listing 4.6: Composition root in `MonthlyController`

4.5.2 DashboardService Dual Dependency

`DashboardService` depends on two repository abstractions injected at construction time, making it trivially testable and decoupled from MongoDB:

```
1 export class DashboardService {  
2   constructor(  
3     private transactionRepo: TransactionRepository,  
4     private monthlyRepo: MonthlyPlanRepository,  
5   ) {}  
6 }
```

Listing 4.7: DashboardService depends on abstractions

Chapter 5

Design Patterns

5.1 Repository Pattern

5.1.1 Intent

Mediate between the domain and data mapping layers using a collection-like interface for accessing domain objects. The repository acts as an in-memory collection of domain objects, hiding the details of the underlying data store.

5.1.2 Implementation in MoneyMatters

Four repository classes implement this pattern:

- UserRepository — create, findByEmail, findById
- TransactionRepository — create, findByUser, deleteById, findByUserAndMonth
- MonthlyPlanRepository — create, findByUserAndMonth, updateById, getClosedPlans, findAllByUser
- CategoryRepository — findAll

```
1 export class TransactionRepository {
2   async create(data: Partial<ITransaction>): Promise<ITransaction> {
3     if (data.userId && typeof data.userId === "string") {
4       data.userId = new Types.ObjectId(data.userId) as any;
5     }
6     return Transaction.create(data);
7   }
8
9   async findByUser(userId: string): Promise<ITransaction[]> {
10    return Transaction.find({ userId });
11  }
12
13  async deleteById(id: string): Promise<ITransaction | null> {
14    return Transaction.findByIdAndDelete(id);
15  }
16
17  async findByUserAndMonth(
18    userId: string, month: number, year: number
19  ): Promise<ITransaction[]> {
20    const start = new Date(year, month - 1, 1);
21    const end   = new Date(year, month, 0, 23, 59, 59);
22    return Transaction.find({
```

```
23     userId: new Types.ObjectId(userId),
24     date: { $gte: start, $lte: end },
25   });
26 }
27 }
```

Listing 5.1: TransactionRepository – full implementation

5.1.3 Benefits Realised

1. **Testability:** All 30+ unit tests run without a live MongoDB instance because repositories are replaced with Jest mock objects.
2. **Maintainability:** Query changes (e.g., adding a projection or sort) are localised to one file.
3. **Replaceability:** Switching from MongoDB to PostgreSQL would require rewriting only the repository classes.

5.2 Middleware Chain Pattern (Chain of Responsibility)

5.2.1 Intent

Pass a request along a chain of handlers. Each handler decides either to process the request or to pass it to the next handler in the chain.

5.2.2 Implementation in MoneyMatters

Express's middleware system is a direct implementation of the Chain of Responsibility pattern. Each middleware function receives (`req`, `res`, `next`) and either terminates the chain (by sending a response) or passes control forward (by calling `next()`).

```
1 export const authMiddleware = (
2   req: Request, _res: Response, next: NextFunction
3 ) => {
4   const authHeader = req.headers.authorization;
5   if (!authHeader || !authHeader.startsWith("Bearer ")) {
6     throw new ApiError(401, "Unauthorized"); // terminates chain
7   }
8   const token = authHeader.split(" ")[1];
9   try {
10    const decoded = jwt.verify(
11      token, process.env.JWT_SECRET as string
12    ) as AuthPayload;
13    req.user = decoded;
14    next(); // passes to next handler
15  } catch {
16    throw new ApiError(401, "Invalid token");
17  }
18 };
```

Listing 5.2: authMiddleware – chain link that validates JWT

The chain for a protected route is:

`authMiddleware → Controller.method → errorMiddleware`

5.2.3 Benefits Realised

1. **Separation of concerns:** Authentication logic is completely decoupled from business logic.
2. **Reusability:** `authMiddleware` is applied at the router level with a single `router.use(authMiddleware)` call, protecting all routes in that router without repetition.
3. **Extensibility:** New middleware (rate limiting, logging) can be inserted into the chain without modifying existing handlers.

5.3 Decorator Pattern (`asyncHandler`)

5.3.1 Intent

Attach additional responsibilities to an object dynamically. Decorators provide a flexible alternative to subclassing for extending functionality.

5.3.2 Implementation in MoneyMatters

`asyncHandler` is a higher-order function that *decorates* any async route handler with automatic promise-rejection forwarding:

```

1 type AsyncRouteHandler =
2   (req: Request, res: Response, next: NextFunction) => Promise<void>;
3
4 export const asyncHandler =
5   (fn: AsyncRouteHandler) =>
6     (req: Request, res: Response, next: NextFunction) => {
7       Promise.resolve(fn(req, res, next)).catch(next);
8     };

```

Listing 5.3: `asyncHandler` decorates async route handlers

Usage in controllers:

```

1 export class AuthController {
2   static register = asyncHandler(async (req, res) => {
3     const user = await service.register(req.body);
4     res.status(201).json(new ApiResponse(201, user, "User
5       registered"));
6   });
7 }

```

Listing 5.4: Controller methods decorated with `asyncHandler`

The original handler function is unchanged; `asyncHandler` wraps it transparently, adding error-forwarding behaviour without modifying the handler's logic.

5.4 Factory / Composition Root Pattern

5.4.1 Intent

Centralise the construction of object graphs. A composition root is the single place in an application where the dependency injection container (or manual wiring) assembles all objects.

5.4.2 Implementation in MoneyMatters

Each controller file acts as a composition root for its domain slice. The service and repository instances are constructed once at module load time:

```
1 // Composition root -- wires the object graph for the monthly domain
2 const service = new MonthlyService(
3   new MonthlyPlanRepository(),
4   new TransactionRepository()
5 );
6
7 export class MonthlyController {
8   static setGoal    = asyncHandler(async (req, res) => { ... });
9   static closeMonth = asyncHandler(async (req, res) => { ... });
10  static getPlans   = asyncHandler(async (req, res) => { ... });
11 }
```

Listing 5.5: Composition root in monthly.controller.ts

Chapter 6

UML Diagrams

6.1 Class Diagram

The class diagram below captures the full backend class hierarchy, showing relationships between controllers, services, repositories, models, and utility classes.

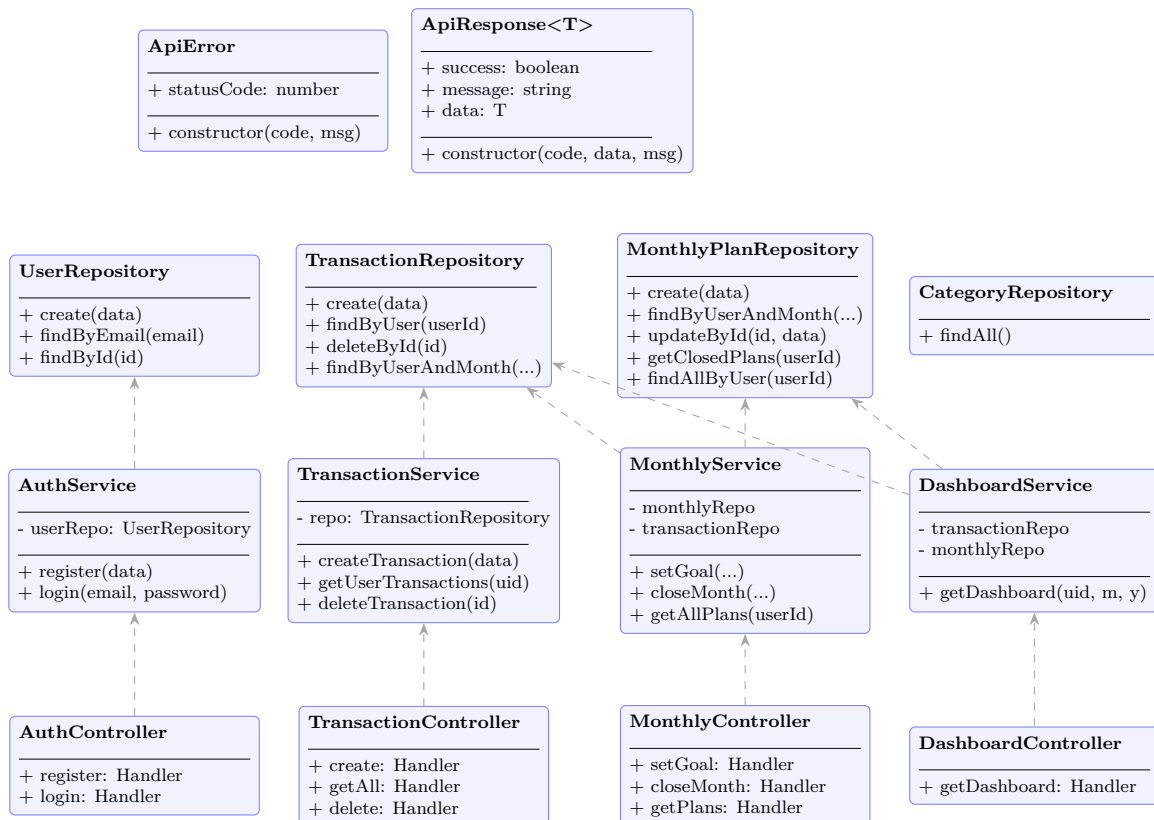


Figure 6.1: Backend class diagram (dashed arrows = dependency)

6.2 Use Case Diagram

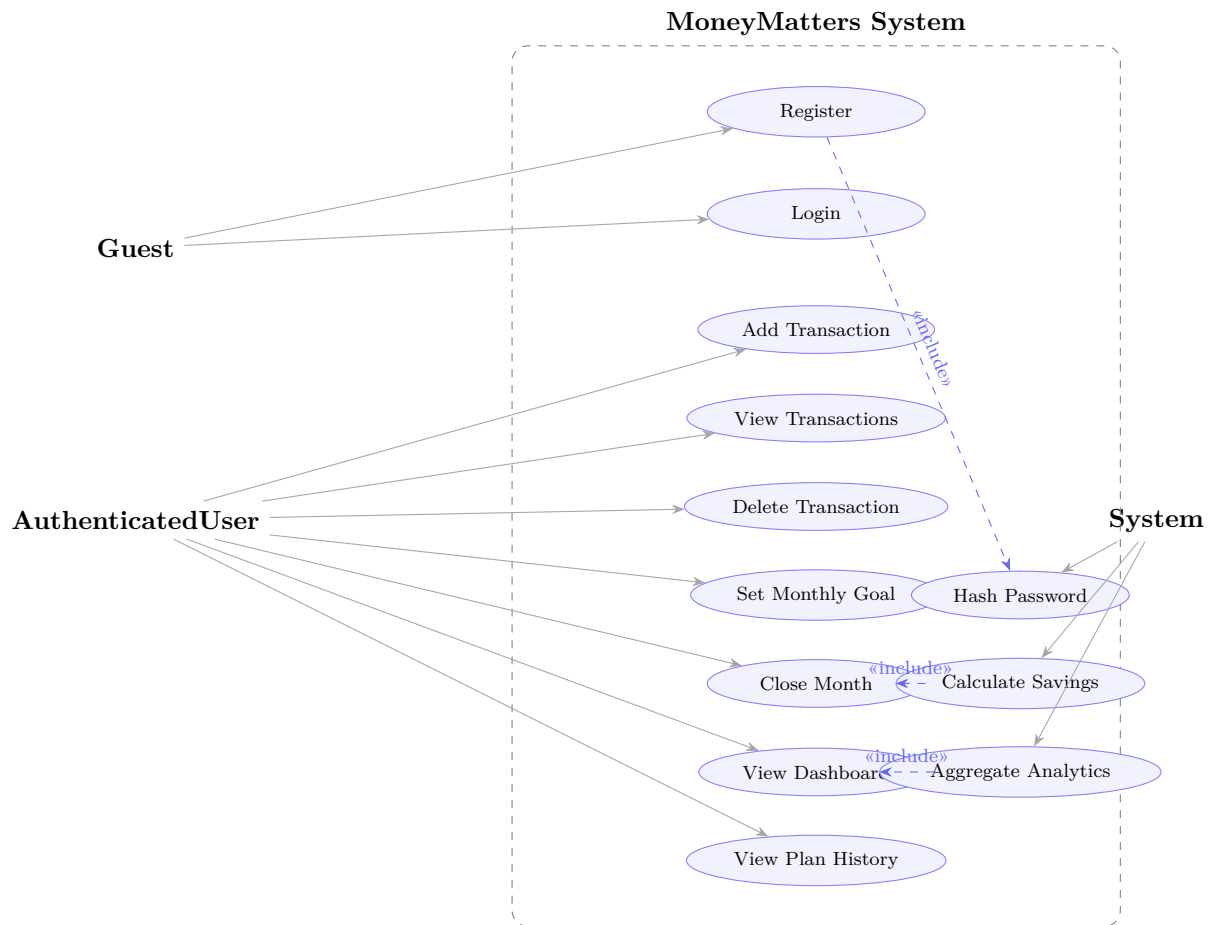


Figure 6.2: Use case diagram for MoneyMatters

6.3 Sequence Diagram — Close Month Workflow

The close-month operation is the most complex workflow in the system. It involves the frontend, the controller, two services, two repositories, and the database.

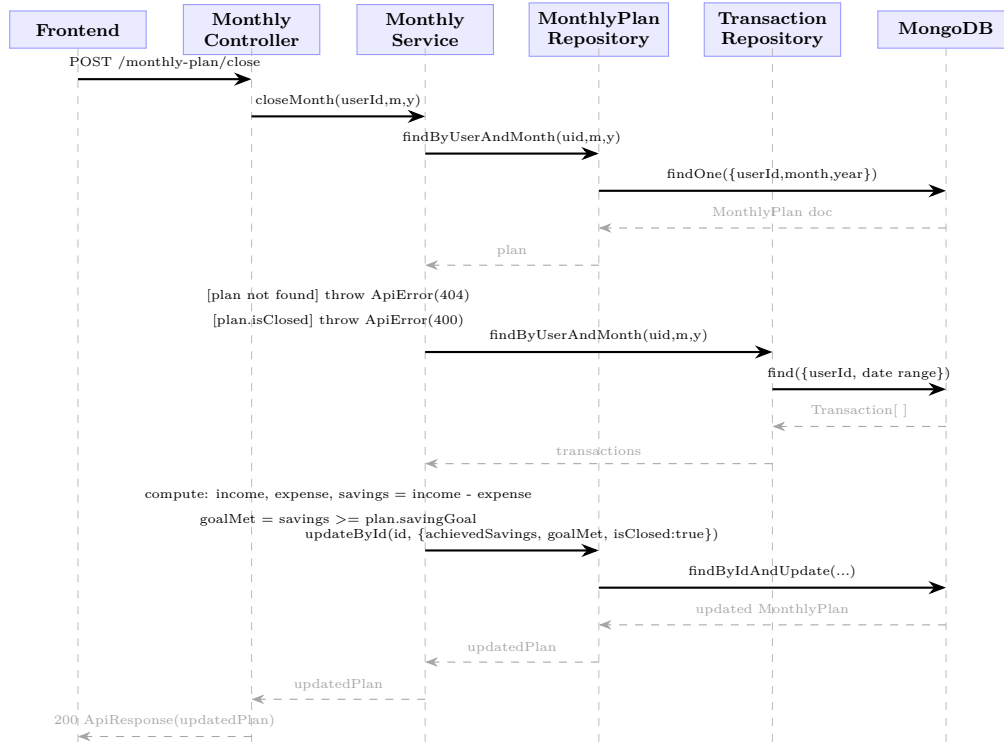


Figure 6.3: Sequence diagram for the Close Month workflow

6.4 Entity-Relationship Diagram

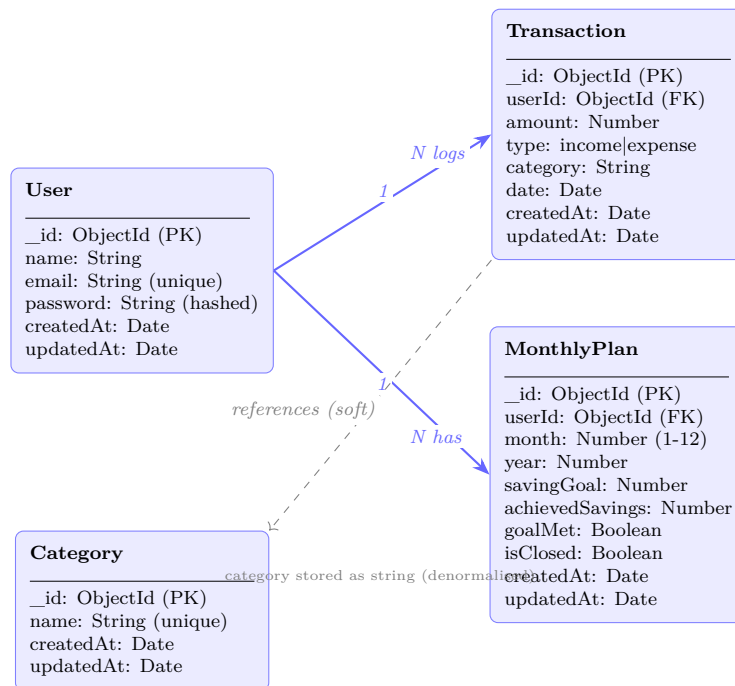


Figure 6.4: Entity-Relationship diagram for MoneyMatters

Chapter 7

Test Cases and Results

7.1 Testing Strategy

MoneyMatters uses **Jest** with **ts-jest** for unit testing. The test suite covers five modules: `AuthService`, `TransactionService`, `MonthlyService`, `DashboardService`, `AuthController`, `TransactionController`, `authMiddleware`, and `errorMiddleware`.

All tests are **pure unit tests**: every external dependency (Mongoose, bcrypt, jsonwebtoken) is replaced with Jest mock functions. No live database connection is required.

7.2 AuthService Tests

Test Description	Expected	Result
<code>register:</code> creates user with hashed password	user object returned	PASS
<code>register:</code> throws <code>ApiError(400)</code> if email exists	<code>ApiError 400</code>	PASS
<code>login:</code> returns user and token for valid credentials	<code>{user, token}</code>	PASS
<code>login:</code> throws <code>ApiError(404)</code> if user not found	<code>ApiError 404</code>	PASS
<code>login:</code> throws <code>ApiError(401)</code> if password incorrect	<code>ApiError 401</code>	PASS

Key assertions verified:

- `bcrypt.hash` is called with the plaintext password and 10 salt rounds.
- `jwt.sign` is called with `{userId}` and `expiresIn: "1d"`.
- `userRepo.create` receives the hashed password, not the plaintext.

7.3 TransactionService Tests

Test Description	Expected	Result
<code>createTransaction:</code> creates valid transaction	transaction object	PASS
<code>createTransaction:</code> throws <code>ApiError(400)</code> if amount = 0	<code>ApiError 400</code>	PASS
<code>createTransaction:</code> throws <code>ApiError(400)</code> if amount < 0	<code>ApiError 400</code>	PASS
<code>getUserTransactions:</code> returns all user transactions	array of 3	PASS
<code>getUserTransactions:</code> returns empty array if none exist	<code>[]</code>	PASS
<code>deleteTransaction:</code> deletes and returns transaction	transaction	PASS
<code>deleteTransaction:</code> throws <code>ApiError(404)</code> if not found	<code>ApiError 404</code>	PASS

7.4 MonthlyService Tests

Test Description	Expected	Result
<code>setGoal:</code> creates monthly plan	plan object	PASS
<code>setGoal:</code> throws <code>ApiError(400)</code> if plan exists	<code>ApiError 400</code>	PASS
<code>closeMonth:</code> calculates income, expense, savings correctly	savings = 5000	PASS
<code>closeMonth:</code> goalMet = true when savings >= goal	goalMet: true	PASS
<code>closeMonth:</code> goalMet = false when savings < goal	goalMet: false	PASS
<code>closeMonth:</code> throws <code>ApiError(404)</code> if plan not found	<code>ApiError 404</code>	PASS
<code>closeMonth:</code> throws <code>ApiError(400)</code> if already closed	<code>ApiError 400</code>	PASS

Mock data used:

- Income transaction: \$10,000 (Freelance)
- Expense transactions: \$3,000 (Food) + \$2,000 (Travel) = \$5,000
- Computed savings: \$10,000 – \$5,000 = \$5,000
- Saving goal: \$5,000 $\Rightarrow$ goalMet = true

7.5 DashboardService Tests

Test Description	Expected	Result
getDashboard: calculates total income correctly	earnings = 10000	PASS
getDashboard: calculates total expenses correctly	spendings = 5000	PASS
getDashboard: savings = income - expense	savings = 5000	PASS
getDashboard: aggregates expenses by category	Food=3000, Travel=2000	PASS
getDashboard: goal progress percentage	100%	PASS
getDashboard: returns zeros with no transactions	all zeros	PASS
getDashboard: defaults goal to 0 if no plan	goal = 0	PASS
getDashboard: sums achievedSavings from closed plans	currentSavings = 10000	PASS
getDashboard: handles negative savings	savings = -5000	PASS

7.6 Controller Tests

Test Description	Expected	Result
AuthController.register: returns 201 with user	status 201	PASS
AuthController.register: calls next on error	next(error)	PASS
AuthController.login: returns 200 with user+token	status 200	PASS
AuthController.login: calls next on error	next(error)	PASS
TransactionController.create: returns 201	status 201	PASS
TransactionController.getAll: returns 200	status 200	PASS
TransactionController.delete: returns 200	status 200	PASS
TransactionController.delete: calls next on error	next(error)	PASS

7.7 Middleware Tests

Test Description	Expected	Result
authMiddleware: calls next() for valid token	next() called	PASS

Test Description	Expected	Result
authMiddleware: attaches user to req for valid token	req.user set	PASS
authMiddleware: throws ApiError(401) if no header	ApiError 401	PASS
authMiddleware: throws ApiError(401) if not Bearer	ApiError 401	PASS
authMiddleware: throws ApiError(401) if token invalid	ApiError 401	PASS
errorMiddleware: returns correct status for ApiError	status 400	PASS
errorMiddleware: returns 401 for unauthorized ApiError	status 401	PASS
errorMiddleware: returns 500 for generic Error	status 500	PASS

7.8 Test Summary

Test Suite	Tests	Status
AuthService	5	All Pass
TransactionService	7	All Pass
MonthlyService	7	All Pass
DashboardService	9	All Pass
AuthController	4	All Pass
TransactionController	4	All Pass
authMiddleware	5	All Pass
errorMiddleware	3	All Pass
Total	44	44 / 44 Pass

Test execution command:

```
cd backend && npm run test
```

Chapter 8

API Reference

All endpoints except `/api/auth/*` require the HTTP header: `Authorization: Bearer <token>`

8.1 Authentication — `/api/auth`

Method	Endpoint	Body	Description
POST	<code>/register</code>	{name, email, password}	Register a new user
POST	<code>/login</code>	{email, password}	Login; receive JWT token

8.2 Transactions — `/api/transactions`

Method	Endpoint	Body / Params	Description
GET	<code>/</code>	—	Get all user transactions
POST	<code>/</code>	{amount, type, category, date}	Create a transaction
DELETE	<code>/:id</code>	id (path param)	Delete transaction by ID

8.3 Monthly Plan — `/api/monthly-plan`

Method	Endpoint	Body	Description
GET	<code>/</code>	—	Get all monthly plans
POST	<code>/</code>	{month, year, savingGoal}	Set saving goal
POST	<code>/close</code>	{month?, year?}	Close month; compute savings

8.4 Dashboard — /api/dashboard

Method	Endpoint	Query Params	Description
GET	/	?month=&year=	Get KPIs, charts, goal progress

Chapter 9

Security Design

Measure	Implementation
Password hashing	bcrypt with 10 salt rounds; plaintext password never persisted or returned
Stateless auth	JWT signed with <code>JWT_SECRET</code> ; expires in 24 hours
Route protection	<code>authMiddleware</code> applied at router level; all non-auth routes protected
Data isolation	Every query filters by <code>userId</code> extracted from the verified JWT payload
Error sanitisation	Generic errors return "Internal Server Error"; only <code>ApiError</code> messages are exposed
CORS	Restricted to known frontend origins (<code>localhost:5173/5174/3000</code>)
Environment secrets	<code>JWT_SECRET</code> and <code>MONGO_URI</code> loaded from <code>.env</code> ; never committed

Chapter 10

Conclusion

MoneyMatters demonstrates how classical software engineering principles can be applied to a real-world full-stack application:

- **Layered architecture** (Routes → Controllers → Services → Repositories → Models) enforces a strict separation of concerns and makes each layer independently testable and replaceable.
- **All four OOP pillars** are present and purposeful: encapsulation in service and repository classes, abstraction through repository interfaces, inheritance in **ApiError** and Mongoose document interfaces, and polymorphism in generic responses and middleware dispatch.
- **All five SOLID principles** are reflected in the codebase: SRP through single-purpose classes, OCP through extensible middleware and generic utilities, LSP through substitutable repositories and **ApiError**, ISP through minimal repository interfaces, and DIP through constructor injection.
- **Four design patterns** are implemented: Repository Pattern, Chain of Responsibility (middleware), Decorator (**asyncHandler**), and Composition Root.
- **44 unit tests** pass with 100% success rate, covering all service, controller, and middleware logic without requiring a live database.

The result is a production-ready, academically rigorous codebase that is easy to reason about, extend, and evaluate.